CS315 · ALGORITHMS ANALYSIS & DESIGN

Lecture 3 Dictionaries & Hash Tables

Midterm summary · Goodrich 2.5

Muharib

What's inside

01 The idea

02 The dictionary ADT

03 The log file

04 Hash function and hash table

05 Hash code maps

06 Compression maps

07 Collisions and chaining

08 Linear probing

09 Search and removal with linear probing

10 Double hashing

11 Performance, load factor and rehashing

12 Review sheet

01 The idea

A dictionary is a collection of items, each with a key that we search by. We want search, insert and delete to be as fast as possible. A hash table turns the key into a cell index of an array, so we go straight to the cell instead of searching.

What the midterm takes from it

A fixed question in the 472 and 451 midterms: insert keys into a hash table of size 9 with chaining, and 451 added double hashing. The 442 midterm asked for the collision-handling methods and for when a hash function is "good".

02 The dictionary ADT

The dictionary ADT models a searchable collection of key-element items. The main operations are searching, inserting, and deleting items. Multiple items with the same key are allowed.

Method	What it does
`findElement(k)`	Returns the element with key `k`, or `NO_SUCH_KEY` if there is none.
`insertItem(k, o)`	Inserts the item `(k, o)`.
`removeElement(k)`	Removes the item with key `k` and returns its element, or `NO_SUCH_KEY` if there is none.
`size(), isEmpty(), keys()`	Information about the dictionary.

Applications: an address book, credit card authorization, and mapping host names to internet addresses.

03 The log file

The simplest dictionary implementation: store the items in an unsorted sequence, in arbitrary order.

Method	Time	Why
`insertItem`	`O(1)`	We insert at the beginning or the end directly.
`findElement`	`O(n)`	In the worst case (the item is not found) we traverse everything.
`removeElement`	`O(n)`	We must find it first.

It suits only small dictionaries, or dictionaries where insertions are common and searches and removals are rare, such as a record of logins.

04 Hash function and hash table

A hash function h maps keys of a given type to integers in a fixed interval [0, N − 1].
Example: h(x) = x mod N is a hash function for integer keys.
The integer h(x) is called the hash value of key x.

A hash table consists of two things: a hash function `h` and an array (the table) of size `N`. The goal is to store item `(k, o)` at index `i = h(k)`.

The lecture example: store `(SSN, Name)` in an array of size `N = 10,000` with `h(x)` = the last four digits of `x`. So `451-229-0004` goes to cell 4.

A hash function is the composition of two functions:

```
Hash code map:      h₁ : keys → integers
```

```
Compression map:    h₂ : integers → [0, N − 1]
```

```
h(x) = h₂(h₁(x))
```

When is a hash function "good"?

When it disperses the keys over the cells in an apparently random, even way, so collisions are few. It should also be fast to compute.

05 Hash code maps

Method	Idea	Suitable for
Memory address	Reinterpret the memory address of the key object as an integer.	Good in general, except for numeric and string keys.
Integer cast	Reinterpret the bits of the key as an integer.	Keys no longer than the integer type (byte, short, int, float).
Component sum	Partition the bits into fixed-length components and sum them.	Numeric keys longer than the integer type (long, double).
Polynomial accumulation	Evaluate a polynomial of the components at a fixed value `z`.	Strings.

$$p(z) = a_0 + a_1 z + a_2 z^2 + \ldots + a_{n-1} z^{n-1}$$

- The component sum is not good for strings, because words with the same letters in a different order (such as tops and pots) give the same sum.

- The choice `z = 33` gives at most 6 collisions on a set of 50,000 English words.
- The polynomial is evaluated in `O(n)` time with Horner's rule.

06 Compression maps

Method	Formula	Condition
Division	$h_2(y) = y \bmod N$	`N` is usually chosen to be a prime.
MAD Multiply, Add and Divide	$h_2(y) = (ay + b) \bmod N$	`a` and `b` are nonnegative integers, and $a \bmod N \neq 0$ must hold.

Why the condition? If $a \bmod N = 0$, every key maps to the same cell `b`. The lecture example:

```
N = 5,  a = 15,  b = 2
```

$h_2(3) = (45 + 2) \bmod 5 = 47 \bmod 5 = 2$

$h_2(7) = (105 + 2) \bmod 5 = 107 \bmod 5 = 2$

```
always 2, because a mod N = 0
```

07 Collisions and chaining

Collisions occur when different elements are mapped to the same cell.

Method	Type	Idea
Chaining	Outside the table	Each cell points to a linked list of the elements mapped there.
Linear probing	**Open addressing**	Place the item in the next available cell.
Double hashing	**Open addressing**	Jump by a step given by a second function `d(k)`.

Chaining is simple, but it requires additional memory outside the table. With open addressing the colliding item is placed in a different cell of the table itself.

Ex Lecture example: chaining

Draw the 7-item hash table resulting from hashing keys 12, 44, 13, 55 and 5, using the hash function h(i) = (3i + 2) mod 7, and assume that collisions are handled by chaining.

Key	12	44	13	55	5
(3i+2) mod 7	3	1	6	6	3

`h(12) = 38 mod 7 = 3` `h(44) = 134 mod 7 = 1`

`h(13) = 41 mod 7 = 6` `h(55) = 167 mod 7 = 6`

`h(5) = 17 mod 7 = 3`

Index	Chain
0	ø
1	44
2	ø
3	12 → 5
4	ø
5	ø
6	13 → 55

A new key is added at the **end** of the chain, as the midterm question asks.

08 Linear probing

If cell `h(k)` is taken we try the next one, and the next, wrapping around from the end of

the table to its start. Each cell we inspect is called a probe.

```
probe j:   (h(k) + j) mod N        for j = 0, 1, 2, …
```

Ex **Lecture example**

h(x) = x mod 13. Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order.

Key	k mod 13	Probes	Cell
18	5	5	**5**
41	2	2	**2**
22	9	9	**9**
44	5	5 → 6	**6**
59	7	7	**7**
32	6	6 → 7 → 8	**8**
31	5	5 → 6 → 7 → 8 → 9 → 10	**10**
73	8	8 → 9 → 10 → 11	**11**

0	1	2	3	4	5	6	7	8	9	10	11	12
		41			18	44	59	32	22	31	73	

Its weakness: colliding items lump together, so later collisions need longer probe sequences. Look at 31: it needed 6 probes.

09 Search and removal with linear probing

findElement(k): we start at cell `h(k)` and probe consecutive cells until one of three things happens: the key is found, an empty cell is found, or `N` cells have been probed without

success.

```
Algorithm findElement(k)
    i ← h(k)
    p ← 0
    repeat
        c ← A[i]
        if c = ø
            return NO_SUCH_KEY
        else if c.key() = k
            return c.element()
        else
            i ← (i + 1) mod N
            p ← p + 1
    until p = N
    return NO_SUCH_KEY
```

The problem with removal: if we empty a cell, a search for an item after it stops there and reports that it is missing. The fix is to replace the removed item with a special marker called AVAILABLE.

Method	What it does
`removeElement(k)`	Search for `k`. If found, replace it with `AVAILABLE` and return its element. Otherwise return `NO_SUCH_KEY`.
`insertItem(k, o)`	Throw an exception if the table is full. Start at `h(k)` and probe until a cell is found that is empty **or** stores `AVAILABLE`, then store the item there.

Example: remove 44 from the lecture table. Cell 6 becomes `AV` (AVAILABLE). A search for 31 starts at 5 and passes cell 6 because it is not empty, then finds 31 at cell 10.

0	1	2	3	4	5	6	7	8	9	10	11	12
		41			18	AV	59	32	22	31	73	

10 Double hashing

Instead of moving one cell at a time, we jump by a step `d(k)` computed by a second function. Each key gets its own step, which reduces the lumping.

```
probe j:   (i + j·d(k)) mod N        j = 0, 1, … , N − 1
```

```
i = h(k)
```

```
d(k) = q − (k mod q)                 q < N,  q is a prime
```

- `d(k)` cannot have zero values.
- `N` must be a prime to allow probing of all the cells.
- The possible values of `d(k)` are `1, 2, … , q`.

Ex Lecture example

N = 13, h(k) = k mod 13, d(k) = 7 − (k mod 7). Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order.

Key	h(k)	d(k)	Probes	Cell
18	5	3	5	**5**
41	2	1	2	**2**
22	9	6	9	**9**
44	5	5	5 → 10	**10**
59	7	4	7	**7**
32	6	3	6	**6**
31	5	4	5 → 9 → 0	**0**
73	8	4	8	**8**

0	1	2	3	4	5	6	7	8	9	10	11	12
31		41			18	32	59	73	22	44		

`44: h = 5 taken, d = 7 − 2 = 5 → (5 + 5) mod 13 = 10`

`31: h = 5 taken, d = 7 − 3 = 4 → 9 taken → (5 + 8) mod 13 = 0`

Ex **Second lecture example**

A hash table of size 11 with double hashing: H(k) = k mod 11, H2(k) = 5 − (k mod 5). Insert 16, 239, 34, 12, 56.

Key	k mod 11	5 − (k mod 5)	Probes	Cell
16	5	4	5	**5**
239	8	1	8	**8**
34	1	1	1	**1**
12	1	3	1 → 4	**4**
56	1	4	1 → 5 → 9	**9**

0	1	2	3	4	5	6	7	8	9	10
	34			12	16			239	56	

Watch the midterm's function

The 451 midterm gave a different second function: `h₂(k) = 1 + (k mod 5)`. The method is exactly the same. Just substitute the function the question gives you.

11 Performance, load factor and rehashing

Case	Time	When
Worst case	$O(n)$	When all the inserted keys collide.
Expected	$O(1)$	For all the dictionary ADT operations.

Load factor: $\alpha = \frac{N}{M}$

N = number of keys inserted, M = size of the table

Expected probes for an insertion (open addressing) = $\frac{1}{1 - \alpha}$

α	0.25	0.5	0.75	0.9
$\frac{1}{1 - \alpha}$	1.33	2	4	10

- In practice hashing is very fast, provided the load factor is not close to 100%.
- Rehashing: when the table fills up, we create a new table of double the size and move the items to it.
- Other applications: small databases, compilers and browser caches.

Mind the symbols: on the load factor slide `N` is the number of keys and `M` is the table size. In the rest of the lecture `N` is the table size.

12 Review sheet

Question	Answer
Log file	insertItem O(1) · findElement O(n) · removeElement O(n)
Hash table	Expected O(1) · Worst O(n)
Collision-handling methods?	Chaining · Linear probing · Double hashing
Linear probing	$(h(k) + j) \bmod N$
Double hashing	$(h(k) + j \cdot d(k)) \bmod N$, $d(k) = q - (k \bmod q)$
MAD	$(ay + b) \bmod N$, $a \bmod N \neq 0$
Removal marker?	AVAILABLE
Load factor	$\alpha = \frac{N}{M}$, probes $= \frac{1}{1 - \alpha}$